

Архиватор на основе алгоритма Хаффмана

Долженко Александра Борисовна

Группа: 25.Б42-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Алгоритм Хаффмана — это метод сжатия без потерь, основанный на построении оптимального префиксного кода для символов входного потока. Он широко применяется в стандартах JPEG, MP3, а также в файловых системах и телекоммуникации.

В рамках учебной практики разработано консольное приложение для сжатия и восстановления файлов по алгоритму Хаффмана. Работа позволяет освоить построение бинарных деревьев, работу с битовыми потоками и управление памятью в языке C.

2. Постановка задачи

Целью работы является реализация консольного приложения для сжатия и восстановления файлов по алгоритму Хаффмана. **Для достижения цели требуется решить следующие задачи:**

1. Выполнить обзор алгоритма Хаффмана и существующих методов сжатия без потерь.
2. Спроектировать архитектуру приложения и формат сжатого файла.
3. Реализовать сжатие и распаковку файлов.
4. Провести эксперименты по оценке степени сжатия и производительности.

3. Обзор

Алгоритм Хаффмана был предложен Дэвидом Хаффманом в 1952 году. Он строит оптимальный префиксный код на основе частот символов: чем чаще символ встречается, тем короче его код. Это гарантирует минимальную среднюю длину кода для заданного распределения частот.

Существуют и другие методы сжатия без потерь: алгоритмы семейства LZ (LZ77, LZ78, LZW), арифметическое кодирование, кодирование длин серий (RLE). Алгоритм Хаффмана уступает им по степени сжатия на текстах, но выигрывает в простоте реализации и скорости работы. В современных архиваторах (gzip, zlib) Хаффман используется в комбинации с LZ77.

Вывод: алгоритм Хаффмана не является лучшим по степени сжатия, но остаётся классическим учебным примером и эффективным инструментом для задач, где важны простота и скорость.

4. Описание предлагаемого решения

Программа состоит из четырёх модулей:

- **bitOperation** — работа с битовым буфером (запись/чтение битов);
- **queue** — очередь с приоритетом (куча) для построения дерева;
- **huffmanTree** — построение дерева и генерация кодов;
- **huffman** — основная логика сжатия и распаковки.

4.1. Алгоритм сжатия

1. Подсчёт частот всех символов в файле.
2. Построение дерева Хаффмана с использованием очереди с приоритетом.
3. Генерация битовых кодов рекурсивным обходом дерева (0 — налево, 1 — направо).
4. Замена символов на битовые коды с упаковкой в байты.
5. Сохранение служебной информации и сжатых данных.

4.2. Формат сжатого файла

- **Маркер** (1 байт, 0xFF) — сигнатура формата.
- **Количество уникальных символов** (4 байта).
- **Таблица частот** — пары «символ (1 байт) + частота (8 байт)».
- **Исходный размер** (8 байт) — для контроля распаковки.
- **Сжатые данные** — битовый поток.

4.3. Алгоритм распаковки

- 1. Чтение заголовка и восстановление таблицы частот.
- 2. Построение дерева Хаффмана по частотам.
- 3. Декодирование битов обходом дерева от корня к листьям.
- 4. Вывод символов до достижения исходного размера файла.

4.4. Технические детали

Для построения дерева используется очередь с приоритетом на основе бинарной кучи, что даёт сложность $O(\log n)$ на операцию. Отдельно обрабатывается случай файла с одним уникальным символом: создаётся фиктивный родительский узел, чтобы избежать вырожденного дерева.

4.5. Обоснование выбора представления таблицы частот

В сжатом файле таблица частот хранится в виде пар «символ (1 байт) — частота (8 байт)», а не в виде полного массива из 256 элементов. Это решение имеет как преимущества, так и ограничения.

Преимущества:

- Для текстовых данных с умеренным алфавитом (английский текст: строчные и заглавные буквы, пробел, знаки препинания, цифры, всего 70–90 уникальных символов) заголовок занимает $70\text{--}90 \times 9 = 630\text{--}810$ байт вместо 2048.
- Размер заголовка пропорционален числу уникальных символов, а не размеру алфавита.
- Формат прост в реализации и однозначен при чтении.

Ограничения:

- Для данных, содержащих все 256 символов с примерно равной частотой (JPEG, случайные данные), заголовок занимает $256 \times 9 = 2304$ байта, что больше фиксированной таблицы из 256 частот (1024 байта).
- Для очень маленьких файлов (менее 1 КБ) заголовок может превышать размер данных, что приводит к увеличению файла.

5. Эксперименты

5.1. Методика

Для оценки степени сжатия и производительности использовались файлы различных типов и размеров: тексты, повторяющиеся данные, нулевые данные, смешанные данные, изображения JPEG, бинарные файлы. Для замеров времени выполнения каждого файла выполнялось 10 прогонов сжатия; вычислялись матожидание и среднеквадратичное отклонение времени.

5.2. Степень сжатия

Таблица 1: Результаты сжатия тестовых файлов

Файл	Исходный (В)	Сжатый (В)	Экономия (%)
Текстовые данные			
text_en_dict_1MB.txt	985,084	551,749	43.99
text_en_dict_100k.txt	102,400	60,417	41.00
text_en_dict_10k.txt	10,240	5,950	41.89
text_58b.txt	58	251	-332.76
Повторяющиеся данные			
repeated_10MB.txt	10,485,760	3,713,782	64.58
repeated_50k.txt	42,000	16,594	60.50
single_char_10K.txt	10,000	1,281	87.19
Нулевые данные			
zero_10MB.bin	10,485,760	1,310,742	87.50
zero_100k.bin	102,400	12,822	87.48
zero_10k.bin	10,240	1,302	87.29
Изображения			
Image_450k.JPG	454,850	451,021	0.84
Image_14MB.jpg	14,568,090	14,547,943	1.00
Бинарные файлы			
bash_binary.bin	1,446,024	1,128,089	21.99
mixed_10MB.bin	10,485,760	8,880,266	15.32
mixed_10K.bin	5,120	7,434	-45.19
random_100k.bin	102,400	104,717	-2.26
random_10k.bin	10,240	12,556	-22.61

Типовые входные данные и результаты экспериментов. Эффективность сжатия существенно зависит от энтропии данных:

- **Очень хорошо сжимаются** данные с низкой энтропией: нулевые данные (до 87.50%), повторяющиеся данные (60–87%), тексты (41–45%).
- **Средне сжимаются** исполняемые файлы (bash_binary.bin: 21.99%): машинный код содержит много нулей (выравнивание секций), повторяющихся инструкций и текстовых строк, что содействует сжатию, но энтропия выше, чем у текста, а количество уникальных символов достигает 256.
- **Плохо сжимаются** уже сжатые данные (Image_14MB.jpg: 1.00%) и случайные данные (random_100k.bin: –2.26%).
- **Ужасно сжимаются** очень маленькие файлы (text_58b.txt: –332.76%): заголовок с таблицей частот превышает размер самих данных, и файл увеличивается в несколько раз.

Таким образом, представление таблицы частот в виде пар «символ — частота» является компромиссом: оно оптимально для текстовых данных, но проигрывает фиксированной таблице на данных с полным алфавитом и высокой энтропией (JPEG, случайные данные).

5.3. Производительность

Таблица 2: Время сжатия файлов ($N = 10$ прогонов), время указано как матожидание $\pm$ СКО.

Файл	Размер (В)	Время, сек
Текстовые данные		
text_en_dict_1MB.txt	985,084	0.215 $\pm$ 0.027
text_en_dict_100k.txt	102,400	0.029 $\pm$ 0.005
text_en_dict_10k.txt	10,240	0.011 $\pm$ 0.003
text_58b.txt	58	0.006 $\pm$ < 0.001
Повторяющиеся данные		
repeated_10MB.txt	10,485,760	0.459 $\pm$ 0.025
repeated_50k.txt	42,000	0.008 $\pm$ 0.001
single_char_10K.txt	10,000	0.005 $\pm$ < 0.001
Нулевые данные		
zero_10MB.bin	10,485,760	0.212 $\pm$ 0.023
zero_100k.bin	102,400	0.008 $\pm$ 0.001
zero_10k.bin	10,240	0.007 $\pm$ 0.001
Смешанные данные		
mixed_10MB.bin	10,485,760	8.157 $\pm$ 0.251
mixed_10K.bin	5,120	0.011 $\pm$ 0.002
Изображения		
Image_14MB.jpg	14,568,090	10.077 $\pm$ 0.559
Image_450k.JPG	454,850	0.364 $\pm$ 0.010
Бинарные файлы		
bash_binary.bin	1,446,024	0.944 $\pm$ 0.040
random_100k.bin	102,400	0.085 $\pm$ 0.003
random_10k.bin	10,240	0.015 $\pm$ 0.003

Выводы: время сжатия растёт с размером файла и увеличением его энтропии. Наибольшая скорость достигается на нулевых данных (10 МВ за 0.21 сек, 50 МВ/сек) и повторяющихся данных (10 МВ за 0.46 сек, 23 МВ/сек), поскольку коды короткие. Наименьшая скорость достигается на случайных данных и больших JPEG (10 МВ за 7–8 сек, 1.3 МВ/сек) из-за глубокого дерева и высокой энтропии.

5.4. Проверка памяти

Valgrind не обнаружил утечек памяти:

```
1 ==13166== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
2 ==13166== All heap blocks were freed -- no leaks are possible
```

6. Заключение

В ходе выполнения работы получены следующие результаты:

- Реализован архиватор на основе алгоритма Хаффмана: сжатие и распаковка файлов, поддержка любых типов данных, обработка краевых случаев.
- Проведены эксперименты: показано, что наилучшее сжатие достигается на данных с низкой энтропией (до 87.5%), реальный текст сжимается на 41–44%, уже сжатые данные не сжимаются.
- Измерена производительность: время сжатия зависит от размера и энтропии, скорость составляет от 1.3 МВ/сек (случайные данные) до 50 МВ/сек (нулевые данные), утечек памяти не обнаружено.

Реализацию можно найти по адресу: https://github.com/DolzhenkoAlexa/Project_huffman